Version Control and Reproducibility

14.33 Research and Communication in Economics

Jon Cohen

September 23, 2026

Agenda

1. Use the command line to navigate a project and run programs
2. Use Git to record and recover project states
3. Use Git while building an individual research project
4. Record the inputs, environment, and commands needed to reproduce a result

Command line interface

A terminal lets you navigate files and run programs

- On macOS: **Terminal** or an installed application such as **iTerm2**
- On Windows: **PowerShell**, **Windows Terminal**, or **Git Bash**
- In an editor: an integrated terminal, such as the terminal in VS Code

```
current folder → enter a command → a program runs → text output
```

Structure of a command

```
ls -la data/
python code/clean.py data/raw/survey.csv
git log --oneline
```

Example	Program	Subcommand	Option	Argument
`ls -la data/`	`ls`	—	`-la`	`data/`
`python code/clean.py data/raw/survey.csv`	`python`	—	—	script and input file
`git log --oneline`	`git`	`log`	`--oneline`	—

A small command-line vocabulary

Goal	Command
Show the current directory	`pwd`
List files	`ls`
Change directory	`cd <path>`
Create a directory	`mkdir <path>`
Read a text file	`cat <file>`
Find the executable for a program	`which <program>`
Show an installed program's version	`<program> --version`
Get help	`<command> --help`

Git’s data model

Organization without version control

```
analysis.py
analysis_new.py
analysis_final.py
analysis_final_fixed.py
analysis_final_final_no_seriously.py
analysis_final_final_2026-09-22.py
```

Version control records project states

```
1 ●  Add a quadratic age term to analysis.py
2 |
3 ●  Correct the treatment variable in analysis.py
4 |
5 ●  Create the initial analysis.py
```

Each **commit** records a project snapshot and an explanation.

Git records history and GitHub stores a remote copy

```
Your computer                          GitHub
Git + local repository  ←————————→   remote Git repository
                                       + website and collaboration tools
```

- **Git** is the version-control software
- **GitHub uses Git** to store another copy of the repository
- Git works locally without GitHub or an internet connection

A Git repository contains a graph



- Each commit records a snapshot
- Each commit identifies its parent
- The history records relationships, not only dates

Each circle represents a recorded state of the project.

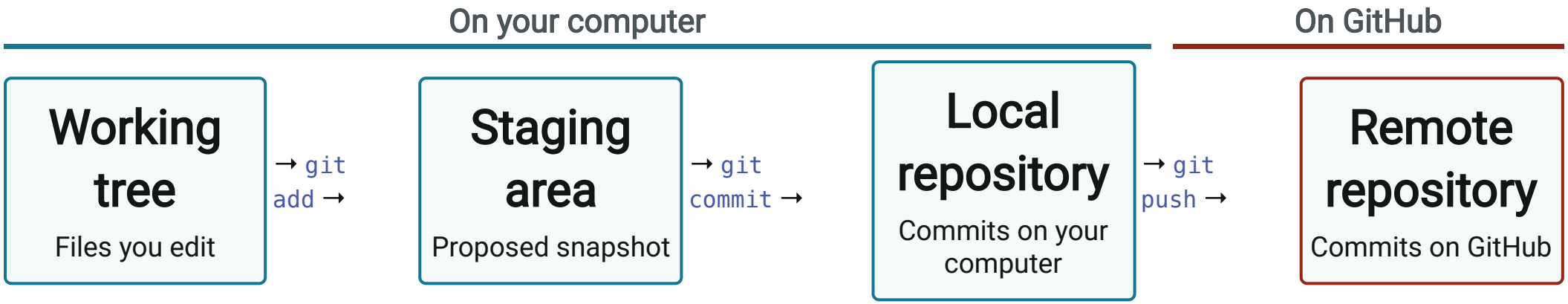
A commit is more than a saved file

```
commit 8e12c4a
Parent:  47d906b
Author:  Student Name
Date:    2026-09-16

    Correct treatment coding
```

Best practice: make each commit one coherent change that you can explain in one sentence.

Commits stay local until you push them



`git pull` brings remote commits into the current local branch and updates the working files.

Exercise: which version will Git commit?

```
# The last commit contains the old model
# The working file now contains model A
git add analysis.py

# Edit the working file again. It now contains model B
git commit -m "Estimate baseline model"
```

Location	Content before `git commit`
Last commit	Old model
Staging area	Model A
Working file	Model B

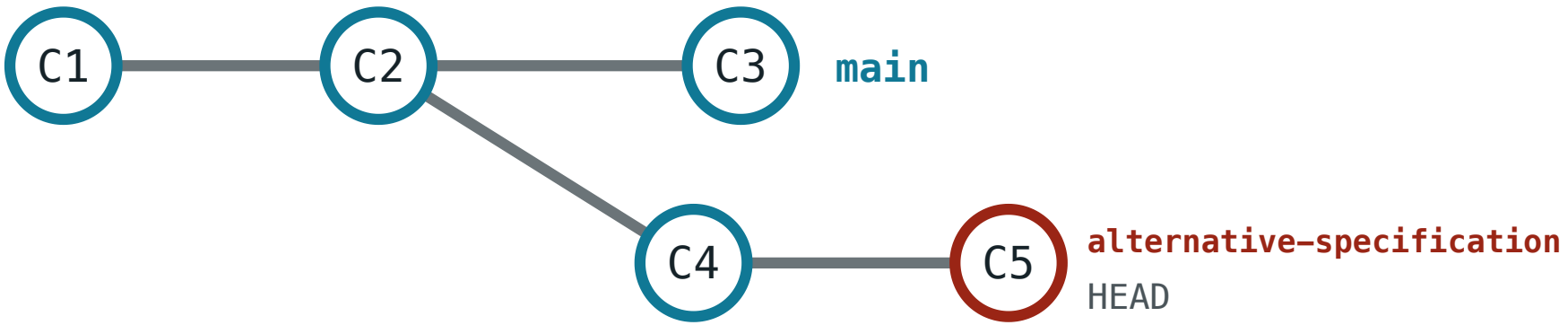
Which model will the new commit contain? What happens to the other two versions?

The correct undo command depends on the state

Where is the unwanted change?	Goal	Command model
Working file only	Discard the local edit	`git restore <file>`
Staging area	Remove it from the proposed commit and keep the edit	`git restore --staged <file>`
A commit others might have	Preserve history and reverse its effect	`git revert <commit>`

First locate the change. Then choose how Git should handle it.

A branch is a movable name for a commit



`main` points to C3; `alternative-specification` points to C5. `HEAD` identifies the current branch.

Exercise: follow HEAD after a commit

```
# Start on main at commit C3
git switch main
git switch -c alternative-specification
# edit analysis.py
git add analysis.py
git commit -m "Estimate quadratic specification"
```

Think about the project immediately after the commit:

- Which commit does `HEAD` identify?
- Which version of `analysis.py` appears if you switch to `main`?
- Does switching to `main` delete the new commit?

A research decision determines what happens to a branch

Decision	Use it when…
Merge	The branch's changes should become part of `main`
Retain for now	The decision is pending and the work is worth revisiting
Delete	The work is rejected and the branch name no longer provides value

If an alternative matters to the paper, preserve its code and result on `main` even when it is not the preferred specification.

A fast-forward merge moves the current branch label

```
Before merge

C1 ← C2 ← C3              main (HEAD)
          ↖ C4            alternative-specification

git switch main                     # destination
git merge alternative-specification

After merge

C1 ← C2 ← C3 ← C4
                  ├── main (HEAD)
                  └── alternative-specification
```

Git moves `main` to C4 without creating a merge commit. The source branch still points to C4.

Exercise: can Git fast-forward this merge?

```
C1 ← C2 ← C3 ← C5        main (HEAD)
          ↖ C4           alternative-specification
```

You run `git merge alternative-specification`.

- Can Git move `main` directly to C4?
- Why might Git need a merge commit?
- When could Git ask you to resolve a conflict?

A divergent merge combines two lines of work

```
Before merge

             C5  main (HEAD)
            ↗
C1 ← C2 ← C3
            ↘
             C4  alternative-specification

After merge

             C5 ──────┐
            ↗         ↓
C1 ← C2 ← C3          C6  main (HEAD)
            ↘         ↑
             C4 ──────┘   alternative-specification stays at C4
```

C6 records the combined result. It has C5 and C4 as parents.

A merge conflict marks a decision Git cannot make

```
Shared starting line:  weight = baseline_weight

main changes it to:   weight = revised_baseline_weight
branch changes it to: weight = alternative_weight
```

Git pauses the merge and marks the conflicting lines in the file.

```
Inspect both versions → choose the intended code
Run checks → stage the resolved file → commit the merge
```

The result may keep one version, both versions, or a new solution.

Exercise: integrate a robustness check without replacing the baseline

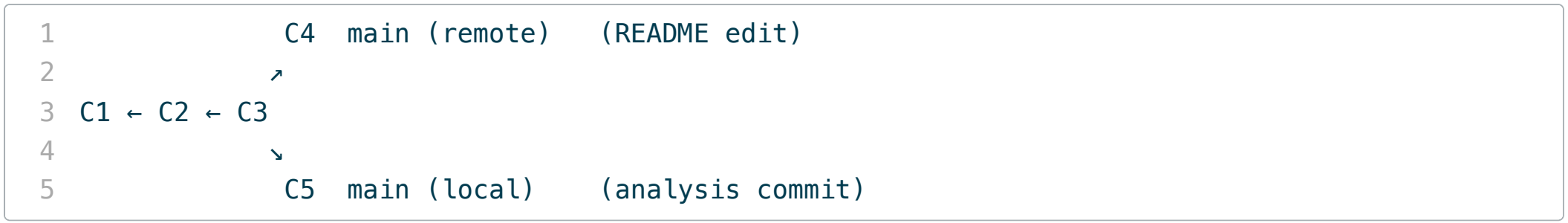
```
main:        baseline weights → primary estimate
alternative: replaces weights → different estimate
decision:    keep primary, report alternative as a check
```

- Would merging the branch as written preserve both estimates?
- What should the code produce before you merge it into `main`?
- What should the research record say about the difference?

Remote commands move commits between repositories

Command	Mental model
`git clone <url>`	Create a local repository and connect it to the remote
`git fetch`	Update your local view of remote commits without changing your working files
`git pull --ff-only`	Fetch, then move your current branch forward only when no histories must be combined
`git push`	Send local commits and advance the corresponding remote branch

Exercise: predict what these remote commands do



Predict the result of each command:

1. Does `git push origin main` move remote `main` to C5, or does Git reject it? Explain from the graph.
2. Does `git pull --ff-only origin main` move local `main` to include C4, or does it stop? Explain from the graph.
3. After `git fetch origin`, does `origin/main` move? Do the project files change? What can `git log --oneline --graph --all` show?

Build a reproducible research project

What must accompany a result?

- The code version: identify the Git commit
- The input data: identify the exact files or release
- The environment and settings: record what the code needs
- The run command and expected output: show how to recreate the result

Exercise: what could make results fail to replicate?

Two researchers check out the same commit. Each has a copy of the input data.

Scenario 1: cannot run

The script stops before producing a result.

Name two plausible explanations.

Scenario 2: different result

The script runs, but the estimate differs.

Name two plausible explanations.

For each scenario, identify what you would record or change to prevent it.

Start with the course repository

```
Course template → your GitHub repository → clone → your computer
```

```
fa26-1433-YOUR-KERBEROS/
├── README.md
├── .gitignore
├── code/
├── data/
└── output/
```

```
git clone https://github.com/YOU/fa26-1433-YOUR-KERBEROS.git
```

Store raw data according to access and size

Data	Git practice
Restricted or confidential	Keep out of Git
Large	Store elsewhere and record its source and version
Small, non-sensitive, and immutable	Commit only if sharing is permitted

Commit extraction code. Store excluded data in an approved location and document its exact version.

Use `.gitignore` for raw data that must stay outside Git.

`.gitignore` keeps selected local files out of commits

Contents of `.gitignore`:

```
# Restricted or large raw data
data/raw/

# Local environments and system files
.venv/
.DS_Store
```

- Commit `.gitignore` so every clone uses the same rules
- Add an ignore rule before you stage the file
- An ignore rule does not remove a file that Git already tracks

Check the project state before recording work

```
$ git status

Untracked files:
  notes/research-question.md

Changes not staged for commit:
  modified: code/analyze.py

Changes to be committed:
  modified: README.md
```

- **Untracked:** exists in the folder but not in a commit or the staging area
- **Modified, not staged:** working file differs from the staged version
- **Staged:** selected for the next commit

Record a coherent project change

```
git status
git add README.md code/analyze.py
git diff --staged
git commit -m "Add baseline analysis"
```

Review the staged diff before committing. The message completes: **This commit will…**

Trace a result back to a code change

```
$ git log --oneline --decorate
3cb418f (HEAD -> main) Add baseline analysis
```

```
Result:  Table 1 baseline estimate
Code:    commit 3cb418f
Data:    survey release 2026-09, crosswalk version 2
Run:     bash run.sh from the project root
Output:  output/estimates.csv
```

Record the commit ID with each important result; identify data separately.

Exercise: predict what happens when you switch branches

```
alternative-specification
  committed version: C4
  working file: edits made after C4, not yet committed

main
  committed version: C2, with a different version of code/analyze.py
```

You want to inspect the committed version on `main`, then return to the edits. What might happen to the working file when you switch branches?

`git stash` pauses edits without advancing the branch

1. Save

`git stash push`

Git records the local edits in a stash

2. Switch

`git switch main`

The working tree can show the committed files from main

3. Return

`git switch alternative-specification`

Go back to the work branch

4. Restore

`git stash pop`

Reapply the edits on that branch

A stash is local and temporary. Apply it after returning to the branch where you paused.

By default, a stash does not include untracked files.

Exercise: switch branches without losing unfinished work

```
Current branch: alternative-specification
Tracked file:   code/analyze.py has uncommitted edits
New file:       notes/weighting.md has never been staged
Goal:           run the committed version on main, then resume both files
```

- Would a default `git stash push` save both files?
- How can you preserve both files before switching branches?
- Why might a checkpoint commit be better if this work is already coherent?

If you need a Python notebook, choose marimo

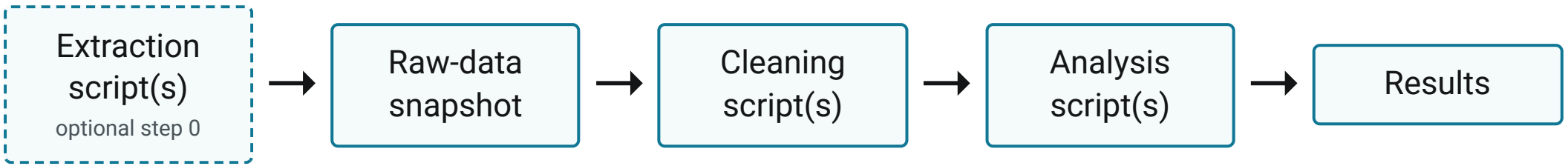
A **notebook** lets you run code in cells beside notes, figures, and results.

	Jupyter	marimo
Saved file	`.ipynb` JSON with code, metadata, and often outputs	A plain `.py` file
Git and coding agents	Changes can be harder to review and edit	Ordinary Python text is easier to diff and edit
Running cells	Old values can remain in memory	Dependent cells update or are marked stale

Course recommendation: use marimo for new interactive Python work.

```
marimo edit notebooks/analysis.py  # explore
python notebooks/analysis.py       # rerun from the command line
```

Script each stage and run it from one entry point



The workflow should have one documented entry point that runs the stages in order.

Use project-relative paths so the workflow can run elsewhere

```
import pandas as pd

# Run the project command from the repository root.
data = pd.read_csv("data/raw/survey.csv")
```

A relative path works when the run command starts from the documented project root.

A shell script records command order

```
#!/usr/bin/env bash
set -euo pipefail
python code/extract.py    # writes data/raw/survey.csv
python code/clean.py      # reads survey and crosswalk; writes data/clean.csv
Rscript code/estimate.R   # writes output/estimates.csv
```

Run this from the project root. The script stops if a command fails.

Exercise: how could you avoid rerunning every command?

Use the script on the previous slide.

1. You changed only `estimate.R`. Which outputs are still valid and can be reused?
2. `crosswalk.csv` changed, but neither script changed. What must be rebuilt before you trust `estimates.csv`?
3. What file relationships would an automation tool need to know to make these decisions?

Make records dependencies between inputs and outputs

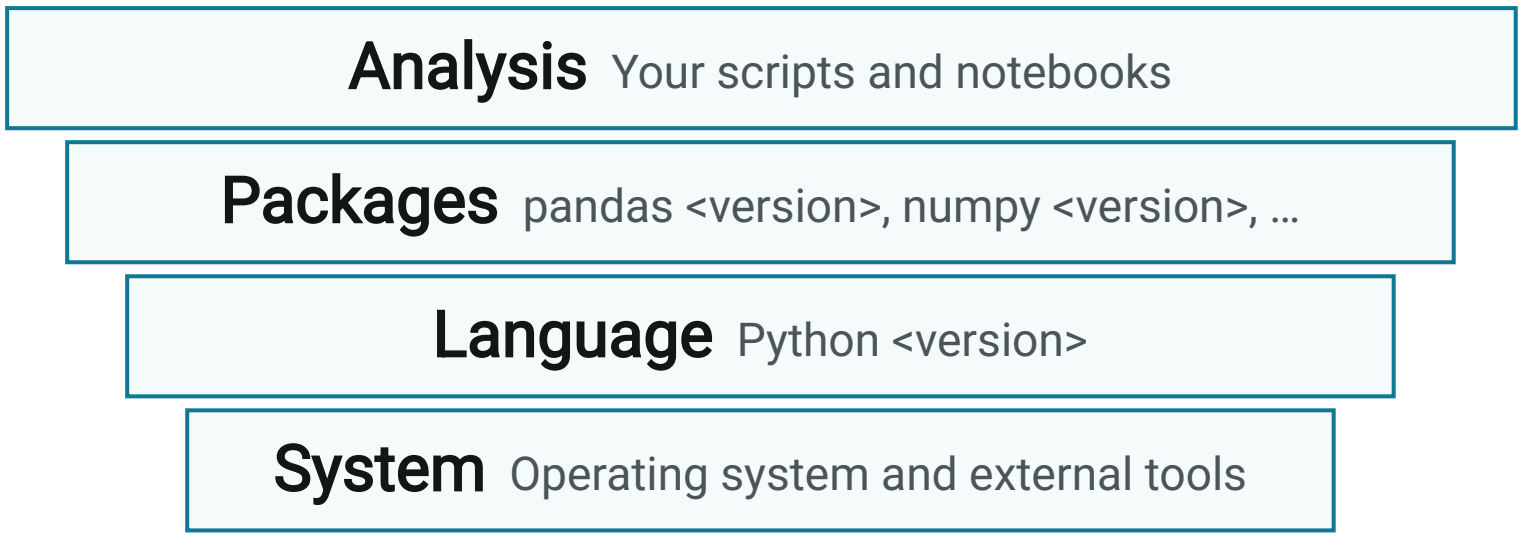
Output	Required inputs
`data/clean.csv`	`data/raw/survey.csv`, `data/raw/crosswalk.csv`, `code/clean.py`
`output/estimates.csv`	`data/clean.csv`, `code/estimate.R`

Make can use declared file dependencies and modification times to decide what to rerun.

Three ways to automate project steps

Option	What it runs	How it decides what to run
One Python script	Python analysis steps	Runs the steps written in the script, in order
Shell script	Commands from Python and other command-line programs	Runs the listed commands, in order
Makefile	Commands from Python and other command-line programs	Rebuilds targets with newer declared prerequisites

Record the computing environment



Different environments can produce different results.

Python example: give each project its own virtual environment

```
Python installation
├── Project A/.venv/ → Project A's Python and packages
└── Project B/.venv/ → Project B's separate Python and packages
```

```
python -m venv .venv
source .venv/bin/activate
```

- Activating `.venv` makes its Python and packages current in the terminal
- Packages installed in Project A's environment do not change Project B's
- `.venv/` is a local directory. Add it to `.gitignore`

Commit the environment recipe, not `.venv/`

```
research-project/
├── .venv/              local installed environment (ignore)
├── requirements.txt    pinned packages and versions (commit)
└── code/
```

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
```

A pinned dependency file helps another researcher recreate `.venv/`.

Control randomness

```
1  import numpy as np
2
3  rng = np.random.default_rng(1433)
4  sample_indices = rng.choice(1000, size=500, replace=True)
```

- Set and record a seed
- Use the same random-number generator
- Document intentional variation across runs

Language-model API calls can change the analysis input

```
Input text → prompt + model call → cleaned or coded data → analysis
```

Source of change	What to record
Generated responses can vary across calls	Retain the exact responses used, subject to data permissions
A provider can update a model	Record the model ID, date, and available version details
A prompt or setting can change	Version the prompt, parameters, and transformation code

Write run instructions in the README

The README should state:

- What the project does and where its inputs come from
- Which environment to create
- Which command runs the analysis
- Where to find the outputs and how to interpret them

For example: from the repository root, run `bash run.sh`.

Summary: keep a clear path from research question to result

Practice	Why it matters to another person and to your future self
Use the command line	Start the project in a known directory and run a documented workflow
Use Git	Identify the code for each result and preserve research decisions
Script and document the workflow	Identify inputs, recreate the environment, and rerun the result

Additional resources

1. **The Missing Semester**, including its **shell** and **Git** lectures
2. **Pro Git**